

A Technical Introduction to Long short-term memory

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$h_t = o_t \odot \tanh(c_t)$$

Section 1

where the initial values are $c_0 = 0$ and $h_0 = 0$ and the operator $\odot$ denotes the Hadamard product (element-wise product). The subscript t indexes the time step.

Section 2

2015: Google started using an LSTM trained by CTC for speech recognition on Google Voice. According to the official blog post, the new model cut transcription errors by 49%. 2016: Google started using an LSTM to suggest messages in the Allo conversation app. In the same year, Google released the Google Neural Machine Translation system for Google Translate which used LSTMs to reduce translation errors by 60%. Apple announced in its Worldwide Developers Conference that it would start using the LSTM for quicktype in the iPhone and for Siri. Amazon released Polly, which generates the voices behind Alexa, using a bidirectional LSTM for the text-to-speech technology. 2017: Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks. Microsoft reported reaching 94.9% recognition accuracy on the Switchboard corpus, incorporating a vocabulary of 165,000 words. The approach used "dialog session-based long-short-term memory". 2018: OpenAI used LSTM trained by policy gradients to beat humans in the complex video game of Dota 2, and to control a human-like robot hand that manipulates physical objects with unprecedented dexterity. 2019: DeepMind used LSTM trained by policy gradients to excel at the complex video game of Starcraft II.

Section 3

Motivation In theory, classic RNNs can keep track of arbitrary long-term dependencies in the input sequences. The problem with classic RNNs is computational (or practical) in nature: when training a classic RNN using back-propagation, the long-term gradients which are back-propagated can "vanish", meaning they can tend to zero due to very small numbers creeping into the computations, causing the model to effectively stop

learning. RNNs using LSTM units partially solve the vanishing gradient problem, because LSTM units allow gradients to also flow with little to no attenuation. However, LSTM networks can still suffer from the exploding gradient problem. The intuition behind the LSTM architecture is to create an additional module in a neural network that learns when to remember and when to forget pertinent information. In other words, the network effectively learns which information might be needed later on in a sequence and when that information is no longer needed. For instance, in the context of natural language processing, the network can learn grammatical dependencies. An LSTM might process the sentence "Dave, as a result of his controversial claims, is now a pariah" by remembering the (statistically likely) grammatical gender and number of the subject Dave. Note that this information is pertinent for the pronoun his and note that this information is no longer important after the verb is.

Section 4

Long short-term memory (LSTM) is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs. Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods. It aims to provide a short-term memory for RNN that can last thousands of timesteps (thus "long short-term memory"). The name is made in analogy with long-term memory and short-term memory and their relationship, studied by cognitive psychologists since the early 20th century. An LSTM unit is typically composed of a cell and three gates: an input gate, an output gate, and a forget gate. The cell remembers values over arbitrary time intervals, and the gates regulate the flow of information into and out of the cell. Forget gates decide what information to discard from the previous state, by mapping the previous state and the current input to a value between 0 and 1. A (rounded) value of 1 signifies retention of the information, and a value of 0 represents discarding. Input gates decide which pieces of new information to store in the current cell state, using the same system as forget gates. Output gates control which pieces of information in the current cell state to output, by assigning a value from 0 to 1 to the information, considering the previous and current states. Selectively outputting relevant information from the current state allows the LSTM network to maintain useful, long-term dependencies to make predictions, both in current and future time-steps. LSTM has wide applications in classification, data processing, time series analysis tasks, speech recognition, machine translation, speech activity detection, robot control, video games, healthcare, energy forecasting.

Section 5

Variants In the equations below, the lowercase variables represent vectors. Matrices W_q and U_q contain, respectively, the weights of the input and recurrent connections, where the subscript q can either be the input gate i , output gate o , the forget gate f or the memory cell c , depending on the activation being calculated. In this section, we are thus using a "vector notation". So, for example, $c_t \in \mathbb{R}^h$ is not just one unit of one LSTM cell, but contains h LSTM cell's units.

Section 6

Applications 2001: Gers and Schmidhuber trained LSTM to learn languages unlearnable by traditional models such as Hidden Markov Models. Hochreiter et al. used LSTM for meta-learning (i.e. learning a learning algorithm). 2004: First successful application of LSTM to speech Alex Graves et al. 2005: Daan Wierstra, Faustino Gomez, and Schmidhuber trained LSTM by neuroevolution without a teacher. Mayer et al. trained LSTM to control robots. 2007: Wierstra, Foerster, Peters, and Schmidhuber trained LSTM by policy gradients for reinforcement learning without a teacher. Hochreiter, Heuesel, and Obermayr applied LSTM to protein homology detection the field of biology. 2009: Justin Bayer et al. introduced neural architecture search for LSTM. 2009: An LSTM trained by CTC won the ICDAR connected handwriting recognition competition. Three such models were submitted by a team led by Alex Graves. One was the most accurate model in the competition and another was the fastest. This was the first time an RNN won international competitions. 2013: Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton used LSTM networks as a major component of a network that achieved a record 17.7% phoneme error rate on the classic TIMIT natural speech dataset. 2017: Researchers from Michigan State University, IBM Research, and Cornell University published a study in the Knowledge Discovery and Data Mining (KDD) conference. Their time-aware LSTM (T-LSTM) performs better on certain data sets than standard LSTM. 2020: Kaplan and McCandlish et al. explored the scaling law of LSTM architecture in language modeling and compared it with Transformers, showing that Transformers scale better than LSTM on text data.